THE VIBE CODING BLUEPRINT

The documentation system for building apps with AI

8 core Markdown templates • user flows • prompts • workflows • checklists

Give your coding agent enough context to spend less time guessing and more time implementing.

A practical, copy-and-adapt guide

This edition contains the eight complete core documents from the supplied text. The source introduces an advanced layer and stops at the beginning of API.md; it does not include the advertised 15+ templates. Added practical prompts and workflow notes are clearly marked.

INTRODUCTION

Start here

Vibe coding makes it incredibly easy to go from an idea to working software. You describe what you want. AI writes the code. You run it. You fix something. You ask AI to change it. Repeat.

This works surprisingly well for small experiments. But as your application becomes more complex, a different problem appears: your AI coding agent starts guessing.

- What a feature should do
- How the UI should look
- How the database should work
- How authentication should work
- Where a file belongs
- How an API should behave

Eventually, different parts of the application start following different assumptions. Documentation gives you and your coding agent a shared source of truth.

How to use this guide

- Copy the relevant template into your repository.
- Replace every placeholder with a real decision or an explicitly unresolved question.
- Start with the smallest set of documents needed for your app.
- Ask the agent to read the relevant files before implementation.
- Validate the behavior and keep the docs aligned with the code.

Templates use generic examples. They are starting points, not requirements to adopt a particular stack or product.

PART 1 / THE CORE IDEA

Context before code

Do not just give AI a prompt. Give AI context, rules, constraints, architecture, examples, and validation.

Stage	Decision to make
Idea → Requirements	What are we building, for whom, and why?
User flows → Design	How do users move through it, and what do they see?
Architecture → Database → API	How does it work, what is stored, and what are the contracts?
Security → Conventions → Testing	What is protected, how is code written, and how is it checked?
Agent instructions → Implementation → Validation	How should the agent work, and what proves completion?

The documentation becomes the shared source of truth for both you and your AI coding agent.

Project structure

```
docs/
PRD.md
DESIGN_SYSTEM.md
ARCHITECTURE.md
DATABASE.md
SECURITY.md
CODE_STYLE.md
TESTING.md
AGENTS.md
```

Placement note (editorial): this reproduces the supplied folder list. Put agent instructions where your coding tool actually discovers them; a root AGENTS.md can point to the documents in docs/. Do not assume docs/AGENTS.md applies to the entire repository.

PART 2 / DOCUMENT MAP

The eight core documents

Document	Main question
PRD.md	What are we building?
DESIGN_SYSTEM.md	What should it look and feel like?
ARCHITECTURE.md	How should the system work?
DATABASE.md	What data exists and how does it relate?
SECURITY.md	What must be protected and how?
CODE_STYLE.md	How should code be written?
TESTING.md	How will we know it works?
AGENTS.md	How should the AI coding agent behave?

Reading the templates

The following sections preserve the supplied fields, rules, examples, and checklists in a compact copyable layout. Markdown heading markers are retained inside templates. Remove placeholder examples that do not apply to your app.

PRODUCT REQUIREMENTS DOCUMENT

01 / PRD.md

Purpose

The PRD defines what you are building, why, who will use it, the core features, the scope, and what success looks like. Your AI should understand the product without needing a 30-minute explanation.

When to create and update it

Create it before implementation. Update it when requirements, features, user flows, or product scope change.

```
# Product Requirements Document

## 1. Product Overview

Product Name: [Name]

One-Line Description: [Explain the product in one sentence.]

Product Type: [SaaS / Mobile App / Marketplace / Internal Tool / etc.]

Primary Platform: [Web / iOS / Android / Desktop]

## 2. Problem

What problem are we solving? [Describe the problem.]

Why does this problem matter? [Explain the importance.]

How are users solving this today? [Current alternatives.]

## 3. Goal

Primary Goal: [What should this product achieve?]

Secondary Goals:

- [Goal]

- [Goal]
```

4. Target Users

User Type 1

Name: [Persona]

Description: [Who are they?]

Needs:

- [Need]
- [Need]

Pain Points:

- [Pain point]
- [Pain point]

5. User Stories

As a [user], I want to [action] so that [outcome].

Examples:

- As a user, I want to create an account so that I can save my data.
- As a user, I want to create a project so that I can organize my work.
- As a user, I want to invite teammates so that we can collaborate.

6. Core Features

Feature 1 - [Feature Name]

Purpose: [What does it do?]

User Flow: [Describe the flow.]

Inputs: [List inputs.]

Outputs: [List outputs.]

Acceptance Criteria:

- [Requirement]
- [Requirement]
- [Requirement]

Edge Cases:

- [Edge case]
- [Edge case]

Feature 2 - [Feature Name]

Purpose: [What does it do?]

User Flow: [Describe the flow.]

Acceptance Criteria:

- [Requirement]
- [Requirement]

7. Non-Goals

The following are intentionally NOT part of this version:

- [Feature]
- [Feature]
- [Feature]

8. MVP Scope

Must Have:

- [Feature]
- [Feature]

Nice to Have:

- [Feature]
- [Feature]

Future:

- [Feature]
- [Feature]

9. Success Metrics

Primary Metrics:

- [Metric]
- [Metric]

Secondary Metrics:

- [Metric]
- [Metric]

10. Constraints

Technical: [Constraint]

Business: [Constraint]

Design: [Constraint]

Timeline: [Constraint]

11. Assumptions

- [Assumption]
- [Assumption]

12. Open Questions

- [Question]
- [Question]

13. Definition of Done

The feature/product is complete when:

- [ ] Requirements are implemented
- [ ] Critical user flows work
- [ ] Tests pass
- [ ] Build succeeds
- [ ] Security requirements are satisfied
- [ ] Documentation is updated

DESIGN SYSTEM

02 / DESIGN_SYSTEM.md

This defines the visual language of your product. Without it, screens can use different fonts, spacing, colors, buttons, or corner styles. A design system gives the agent a consistent visual vocabulary.

```
# Design System
```

1. Design Direction

```
Style: [Minimal / Editorial / Premium / Playful / Technical / etc.]
Mood: [Describe the overall feeling.]
Design Principles:
- [Principle]
- [Principle]
- [Principle]
```

2. Brand

```
Brand Name: [Name]
Logo: [Rules]
Brand Voice: [Description]
```

3. Color Palette

```
Primary: #HEX
Secondary: #HEX
Background: #HEX
Surface: #HEX
Text: #HEX
Muted Text: #HEX
Border: #HEX
Success: #HEX
Warning: #HEX
Error: #HEX
```

4. Typography

Font Family: [Font]

Heading Font: [Font]

Body Font: [Font]

H1: [Size / Weight / Line Height]

H2: [Size / Weight / Line Height]

H3: [Size / Weight / Line Height]

Body: [Size / Weight / Line Height]

Small: [Size / Weight / Line Height]

5. Spacing

Base Unit: [Example: 4px]

Spacing Scale: 4px / 8px / 12px / 16px / 24px / 32px / 48px / 64px / 96px

6. Border Radius

Small: [Value]

Medium: [Value]

Large: [Value]

Pill: [Value]

7. Shadows

Small: [Rule]

Medium: [Rule]

Large: [Rule]

8. Buttons

Primary Button: [Rules]

Secondary Button: [Rules]

Danger Button: [Rules]

Disabled: [Rules]

Loading: [Rules]

9. Inputs

Default: [Rules]

Focus: [Rules]

Error: [Rules]

Disabled: [Rules]

Placeholder: [Rules]

10. Cards

[Define card padding, border, radius, shadow.]

11. Navigation

Header: [Rules]

Sidebar: [Rules]

Mobile Navigation: [Rules]

12. States

Loading: [Rules]

Empty: [Rules]

Error: [Rules]

Success: [Rules]

Disabled: [Rules]

13. Icons

Icon Library: [Library]

Default Size: [Size]

Icon Style: [Outline / Filled / Rounded]

14. Responsive Design

Mobile: [Rules]

Tablet: [Rules]

Desktop: [Rules]

Breakpoints:

Mobile: [Range]

Tablet: [Range]

Desktop: [Range]

15. Accessibility

- Use semantic HTML
- Maintain sufficient contrast
- Support keyboard navigation
- Provide visible focus states
- Provide accessible labels
- Do not rely only on color

ARCHITECTURE

03 / ARCHITECTURE.md

This defines the technical structure: where everything goes, how components communicate, and why the architecture was chosen.

```
# Architecture

## 1. System Overview

[Describe the system at a high level.]

## 2. Architecture Pattern

Pattern: [Monolith / Modular Monolith / Microservices / Serverless / etc.]

Reason: [Why?]

## 3. Technology Stack

Frontend: [Technology]

Backend: [Technology]

Database: [Technology]

Authentication: [Technology]

Storage: [Technology]

Hosting: [Technology]

Payments: [Technology]

Email: [Technology]

Analytics: [Technology]

Other Services: [List]

## 4. System Components

### Frontend

Responsibilities:

- [Responsibility]

- [Responsibility]

### Backend

Responsibilities:

- [Responsibility]

### Database

Responsibilities:

- [Responsibility]
```

5. Architecture Diagram

[Insert diagram here.]

Example: User → Frontend → API → Service Layer → Database

6. Data Flow

1. User submits form
2. Frontend validates input
3. Request is sent to API
4. API authenticates user
5. Service validates business rules
6. Database is updated
7. Response is returned
8. UI updates

7. Project Structure

src/

components/

pages/

services/

hooks/

utils/

types/

middleware/

config/

8. Dependency Rules

- [Rule]
- [Rule]
- [Rule]

Example: UI components should not directly access the database.

9. Key Technical Decisions

Decision: [Decision]

Context: [Why did we need to decide?]

Alternatives: [Alternatives]

Reason: [Why this approach?]

10. Performance Considerations

- [Requirement]
- [Requirement]

11. Scalability Considerations

- [Requirement]
- [Requirement]

12. Failure Scenarios

What happens if:

- Database is unavailable?
- API times out?
- Third-party service fails?
- Authentication fails?

Document expected behavior.

DATABASE

04 / DATABASE.md

This is the source of truth for your data model. AI should not randomly invent tables, relationships, fields, or naming conventions.

```
# Database

## 1. Database Engine

Database: [PostgreSQL / MySQL / SQLite / MongoDB / etc.]
Hosting: [Provider]

## 2. Naming Conventions

Tables: [Convention]
Columns: [Convention]
Primary Keys: [Convention]
Foreign Keys: [Convention]
Timestamps: [Convention]

## 3. Tables

### users
Purpose: [Description]
Columns: id, email, name, created_at, updated_at
### projects
Purpose: [Description]
Columns: id, user_id, name, description, status, created_at, updated_at

## 4. Relationships

users → projects: One user can have many projects.
projects → tasks: One project can have many tasks.
[Define the tasks table if this example applies to your product.]

## 5. Constraints

Required:
- [Field]
Unique:
- [Field]
Foreign Keys:
- [Relationship]
```

6. Indexes

Index: [Column]

Reason: [Reason]

7. Data Validation

- [Rule]
- [Rule]

8. Soft Delete

Does the system use soft deletion? [Yes / No]

If yes: [Explain implementation.]

9. Auditing

Do we record:

- Created by?
- Updated by?
- Deleted by?
- Activity history?

[Rules]

10. Sensitive Data

Sensitive fields:

- [Field]

Protection: [Rules]

11. Migration Rules

Schema changes must:

1. Be documented
2. Have a migration
3. Be tested
4. Avoid breaking existing production data

12. Seed Data

[Describe development/demo seed data.]

SECURITY

05 / SECURITY.md

Security is a project requirement. Define authentication, authorization, secrets, API protection, data security, and upload rules before implementation.

```
# Security Guidelines

## 1. Authentication

Authentication Provider: [Provider]

Methods:

- Email/password
- OAuth
- Magic link
- [Other]

## 2. Authorization

Roles: Admin, User, [Role]

Permissions:

### Admin

- [Permission]

### User

- [Permission]

Authorization must be enforced server-side.

## 3. Secrets

- Never hard-code credentials
- Never commit production secrets
- Never expose private secrets to the client
- Use environment variables
- Rotate compromised credentials
- Use appropriate secret storage

## 4. Environment Variables

Public: [Variables]

Private: [Variables]
```

5. API Security

Input validation: [Rules]

Rate limiting: [Rules]

CORS: [Rules]

Authentication: [Rules]

Authorization: [Rules]

6. Data Protection

Sensitive data: [List]

Encryption: [Rules]

Retention: [Rules]

Deletion: [Rules]

7. Logging

Never log:

- Passwords
- API keys
- Tokens
- Sensitive personal information

Log:

- Authentication events
- Important actions
- Errors
- Security events

8. File Uploads

Allowed file types: [List]

Maximum size: [Limit]

Validation: [Rules]

Storage: [Location]

9. Security Checklist

- [ ] Authentication implemented
- [ ] Authorization implemented
- [ ] Server-side validation
- [ ] Secrets protected
- [ ] Sensitive data protected
- [ ] Rate limiting considered
- [ ] File uploads validated
- [ ] Errors do not expose sensitive details
- [ ] Dependencies reviewed

CODE STYLE & CONVENTIONS

06 / CODE_STYLE.md

Keep generated code consistent across the project. Reuse established patterns and make focused, readable changes.

```
# Code Style & Conventions

## 1. General Principles

- Prefer readable code
- Keep functions focused
- Avoid unnecessary abstractions
- Reuse existing components
- Reuse existing utilities
- Prefer consistency over novelty

## 2. Naming

Components: [Convention]
Functions: [Convention]
Variables: [Convention]
Constants: [Convention]
Files: [Convention]
Database fields: [Convention]
API routes: [Convention]

## 3. Folder Structure

[Describe rules.]

## 4. Imports

- [Rule]
- [Rule]

## 5. Components

Components should:
- Have a single responsibility
- Prefer reusable patterns
- Avoid unnecessary duplication
```

6. Functions

Functions should:

- Do one logical thing
- Have clear names
- Avoid unnecessary complexity

7. Error Handling

Use: [Pattern]

Do not: [Pattern]

8. Comments

Comments should explain:

- Why something exists
- Important tradeoffs
- Non-obvious behavior

Avoid comments that simply repeat the code.

9. Logging

Use: [Logging library]

Rules:

- [Rule]
- [Rule]

10. Formatting

Formatter: [Tool]

Linter: [Tool]

Type Checking: [Tool]

11. Do

- Reuse existing patterns
- Keep changes focused
- Follow project conventions
- Remove dead code

12. Don't

- Create duplicate utilities
- Introduce unnecessary dependencies
- Ignore existing architecture
- Change unrelated files

TESTING STRATEGY

07 / TESTING.md

Define how the application will be validated. The agent should not stop at “the code looks correct.” Test the application and verify critical user journeys.

```
# Testing

## 1. Testing Goals

Critical functionality:

- [Feature]

- [Feature]

- [Feature]

## 2. Test Types

### Unit Testing

Test utility functions, business logic, and components where appropriate.

### Integration Testing

Test APIs, database interactions, authentication flows, and service integrations.

### End-to-End Testing

Test critical journeys:

- Sign up

- Sign in

- Create project

- Complete checkout

- [Other critical flows]

## 3. Testing Tools

Unit: [Tool]

Integration: [Tool]

E2E: [Tool]

## 4. Test File Structure

tests/

unit/

integration/

e2e/
```

5. Critical User Flows

Flow: [Name]

Steps:

1. [Step]

2. [Step]

3. [Step]

Expected result: [Result]

6. Coverage

Target: [Practical target]

Prioritize:

- Critical business logic

- Authentication

- Payments

- Data mutations

- Important user journeys

7. Commands

Test: [test command]

Lint: [lint command]

Build: [build command]

Type Check: [typecheck command]

8. Definition of Done

- [ ] Tests pass

- [ ] Build passes

- [ ] Lint passes

- [ ] Type checking passes

- [ ] Critical flow verified

- [ ] No blocking errors

AI CODING AGENT INSTRUCTIONS

08 / AGENTS.md

Turn project documentation into operational instructions. Use the rules mechanism supported by your coding environment and keep these principles centralized.

```
# AI Coding Agent Instructions

## 1. Project Context

Project: [Name]

Purpose: [Description]

Primary Users: [Users]

## 2. Source of Truth

Before significant changes, read relevant documentation inside /docs.

Core documentation:

- PRD.md
- DESIGN_SYSTEM.md
- ARCHITECTURE.md
- DATABASE.md
- SECURITY.md
- CODE_STYLE.md
- TESTING.md

Do not contradict these documents without first identifying the conflict.

## 3. General Rules

- Understand existing code before modifying it
- Reuse existing components and utilities
- Follow the design system
- Follow the architecture
- Follow database conventions
- Follow security requirements
- Follow coding conventions
- Do not invent business requirements
- Do not expose secrets
- Avoid unnecessary dependencies
- Avoid unrelated changes
```

4. Planning

Before implementing a non-trivial task:

1. Inspect relevant documentation
2. Inspect relevant code
3. Identify affected files
4. Identify dependencies
5. Identify database/API changes
6. Identify tests required
7. Create a concise implementation plan

5. Implementation

- Make focused changes
- Reuse established patterns
- Preserve existing behavior
- Avoid unnecessary rewrites
- Keep code maintainable

6. Validation

After implementation:

1. Run tests
2. Run linting
3. Run type checking
4. Run the build
5. Review changed files
6. Check for unintended changes

7. Documentation

Update documentation when implementation changes:

- Product behavior
- Architecture
- Database schema
- API contracts
- Security rules
- Coding conventions

8. Conflict Handling

If requirements conflict:

Do not guess. Identify the conflict and ask for clarification.

9. Completion Criteria

A task is complete only when:

- Requirements are satisfied
- Tests pass
- Build succeeds
- No unintended files changed
- Security rules are respected
- Documentation is updated when necessary

PART 3 / SUPPLIED MATERIAL

The advanced documentation layer

For larger projects, the source introduces additional documentation. It includes UX_FLOWS.md and begins API.md, but the API template is incomplete.

09 / UX_FLOWS.md

Document the actual journeys: Landing page → Sign up → Onboarding → Dashboard → Create project → Project details → Invite team → Complete project.

```
# UX Flows
```

Flow 1 - User Registration

```
1. User opens signup page
2. User enters email
3. User creates password
4. Validation runs
5. Account is created
6. User is redirected to onboarding
Success: [Expected result]
Failure: [Expected behavior]
```

Flow 2 - Create Project

```
1. User clicks Create Project
2. Modal opens
3. User enters name
4. User submits
5. API validates data
6. Project is created
7. User is redirected
Loading State: [Behavior]
Error State: [Behavior]
Empty State: [Behavior]
```

10 / API.md

The source describes this as the source of truth for API contracts and starts an unfinished GET endpoint. A usable starter is provided on the next page as an editorial addition.

EDITORIAL ADDITION / NOT IN THE SUPPLIED TEMPLATE

API contract starter

```
# API Documentation
```

Conventions

```
Base URL: [URL]
Authentication: [Method]
Versioning: [Strategy]
Error format: [Shape]
Pagination: [Method]
Rate limits: [Limits]
```

Endpoint - [METHOD] /[resource]

```
Purpose: [Action]
Required permission: [Role or ownership rule]
Path parameters: [Names, types, constraints]
Query parameters: [Names, types, defaults]
Request body: [Schema and example]
Success response: [Status, schema, example]
Error responses: [Status codes and meaning]
Side effects: [Writes, notifications, external calls]
Idempotency: [Duplicate request behavior]
Validation: [Server-side rules]
Tests: [Success, invalid input, unauthenticated, forbidden]
```

Keep contracts testable

Specify real types, allowed values, and status codes. Separate authentication from ownership checks. Define what happens on retries and partial failures before connecting payments or other external services.

EDITORIAL ADDITION / PRACTICAL WORKFLOW

Copy-and-use AI prompts

01 / Turn an idea into documentation

```
Read the existing repository and relevant docs.
My idea: [idea]. My users: [users]. Constraints: [constraints].
Draft a PRD with MVP scope, non-goals, user stories, acceptance criteria,
and edge cases. Mark assumptions and unresolved decisions explicitly.
Do not choose a new stack or implement code yet.
```

02 / Implement one feature

```
Implement [feature] using [relevant documents]. Inspect existing code
first. Identify affected files and a concise plan. Reuse current patterns.
Respect scope and permissions. Verify the acceptance criteria, run the
applicable checks, and update docs when behavior changes. Report what
changed, what you tested, and any unresolved limitations.
```

03 / Fix a bug

```
Observed behavior: [what happens]. Expected behavior: [what should happen].
Reproduction steps: [steps]. Inspect the affected flow and determine the
root cause. Make a focused fix. Verify the reproduction no longer fails
and check adjacent behavior. Do not rewrite unrelated features.
```

04 / Review before release

```
Review the changed files against the PRD, architecture, API contracts,
and security rules. Check permissions, validation, failure states, and
data migrations. Report concrete findings with impact and file locations.
Separate verified issues from questions. Do not claim checks passed
unless they were run.
```

EDITORIAL ADDITION / WORKFLOW & CHECKLISTS

Build, validate, maintain

Before implementation

[ ] Product problem and target user are clear

[ ] MVP and non-goals are written

[ ] Critical flows include loading, empty, success, and error states

[ ] Architecture, data model, and API decisions agree

[ ] Security and ownership rules are explicit

[ ] Agent instructions point to the correct documents

During implementation

• Build one coherent feature at a time.

• Give the agent file references and acceptance criteria.

• Resolve meaningful conflicts rather than silently choosing a new requirement.

• Review migrations and destructive changes before running them.

• Keep secrets out of prompts, repositories, screenshots, and logs.

Before delivery

[ ] Applicable tests, lint, type checks, and build completed

[ ] Critical user journeys manually verified where needed

[ ] Permissions, invalid inputs, and failures checked

[ ] Changed files reviewed for unintended edits

[ ] Docs reflect the implemented behavior

[ ] Unrun checks and remaining limitations recorded

Keep the system lightweight

Document decisions that affect implementation. Avoid duplicate rules and giant stale files. Tie requirements to verifiable outcomes, and revise docs whenever a real product or technical decision changes.

Your next step

Create the PRD, fill the smallest useful set of supporting documents, and ask your agent to implement one scoped feature against clear acceptance criteria.